

FOL: A Functional Object Lisp

Frank Adrian

Ancar Technology

Olathe, KS, USA

frank.adrian314159@gmail.com

Abstract

FOL (Functional Object Lisp) synthesizes CLOS object orientation with persistent data structures, demonstrating that these paradigms are synergistic rather than competing. While Lisp allows for a wide variety of programming styles, the gap between the CLOS-centric imperative model and the Clojure-centric functional model remains significant. FOL bridges this by providing a CLOS implementation where every object is internally represented by a persistent data structure, allowing researchers and developers to leverage the best of both worlds: the extensibility of the MOP and the versioning power of persistent state. We present a design where persistent slot storage is enabled via a CLOS metaobject protocol (MOP) adaptation, utilizing a hybrid native-slot/overflow-trie layout ($T = 8$) to preserve $O(1)$ access for the common case. We further introduce a four-level category-based specificity ordering for predicate dispatch that provides a decidable, predictable alternative to logical subsumption. Our evaluation indicates that while functional updates incur significant micro-overhead (33–447 $\times$), this cost effectively amortizes in non-trivial workloads. In a 4,096-gate logic simulation, FOL reaches near-parity (1.27 $\times$) with native Common Lisp and, crucially, reduces memory pressure by 0.59 $\times$ via structural sharing, suggesting that the “cost of persistence” becomes a modular advantage at scale.

CCS Concepts

• **Software and its engineering** $\rightarrow$ **Functional languages; Object oriented languages; Extensible languages.**

Keywords

Functional programming, object-oriented programming, Lisp, persistent data structures, CLOS, MOP, transducers

FOL explores the synthesis of Common Lisp-style object orientation (CLOS) and Clojure-style immutable data structures. We find that multiple dispatch over persistent state enables architectural patterns that are more natural than in either parent paradigm alone. While CLOS offers powerful method combinations and a metaobject protocol (MOP), its root in mutable state complicates versioned history and transactional integrity. Conversely, Clojure’s immutable model lacks the introspective extensibility provided by the MOP, requiring externalized state management (atoms, refs) that sits outside the object hierarchy.

Immutable objects simplify concurrency by eliminating synchronization and manual locking. FOL adapts the CLOS MOP to use persistent data structures for slot storage. While Racket and records in ML-family languages offer similar guarantees, FOL’s contribution is applying persistence to full CLOS with method combinations, slot-missing overflow, and lazy schema migration. We implemented FOL as a standalone Lisp to enforce persistence invariants

that library-based approaches in Common Lisp cannot easily guarantee. Library approaches typically rely on discipline (avoiding `setf`) or macros that wrap slot access; by implementing a custom metaclass that intercepts the MOP’s low-level slot accessors, we ensure that persistence is a property of the object itself, not the calling code.

The synthesis of immutability and CLOS makes several patterns more ergonomic:

- **Versioned objects:** Producing instances of new schemas safely while preserving the original state.
- **Event sourcing:** Using `:` around methods on `assoc` to build audit logs and change histories automatically.
- **Observable notifications:** Routing change events via predicate dispatch without manual `cond` cascades.

These patterns leverage the fact that every functional update in FOL is a first-class event that can be intercepted, validated, and transformed by the object’s metaclass.

1 Language Design

1.1 Core Philosophy

We follow Driscoll et al. [9] in using *persistent* to mean preserving previous versions via structural sharing. FOL’s design commitments are: (1) **Immutability by default** for all objects and collections; (2) **Structural sharing** to amortize update costs; (3) **Object identity** distinct from value equality; (4) **Generic functions** with de-structuring; and (5) **MOP-integrated persistence**.

1.2 Identity, Syntax, and Persistence

Following Hickey’s epochal time model [14], FOL distinguishes between the *identity* of a value (which never changes) and the *identity* of an entity (which may progress through a sequence of values). Reference identity via `eq` refers to a specific version of an object; structural value equality via `=` compares the contents of two versions. Syntactically, FOL adopts Clojure’s literal syntax: `[1 2 3]` for vectors, `{:a 1}` for maps, and `#[1 2]` for sets. User-defined classes, declared via `defclass`, inherit from `<persistent-object>` by default. Functional updates via `assoc` produce new instances while preserving the original, ensuring that “modification” is always a non-destructive transformation of the reference.

```
(defclass <person> [] [[name] [age]])  
(def alice (make <person> :name "Alice" :age 30))  
(def older-alice (assoc alice :age 31))
```

Slot storage uses a hybrid strategy: objects with ≤ 8 slots use native CLOS slots for speed, while wider objects overflow into a persistent vector trie ($T=8$). This threshold keeps $\approx 80\%$ of classes on the all-native fast path while preserving $O(1)$ access for small objects and $O(1) + O(\log n)$ for larger ones.

2 Architecture and Implementation

2.1 Collection Implementation

FOL implements nineteen persistent collection classes including **Vectors** (32-way branching persistent tries), **Maps** (HAMTs), **Sets** (HAMTs), **Sorted collections** (B-trees), and **Lists** (persistent cons cells). HAMT lookup is only $1.3\times$ slower than CL gethash for the common case; however, while a hash table exhibits $O(1)$ average-case performance with occasional $O(N)$ rehash spikes, the HAMT provides $O(\log_{32} N)$ guaranteed performance, which for a billion-element map is effectively a five-hop constant-time array access. The B-tree implementation for sorted collections handles 64-way branching to minimize cache misses during traversal.

2.2 Runtime Representations

Most runtime representations other than collections and user-defined objects utilize a Common Lisp base. Primitives (`<bool>`, `<string>`, the numeric tower, `<symbol>`) are unwrapped CL values; functions with their closures are CL functions; the package system and error handling derive from CL. Garbage collection is inherited—old versions of user-defined objects are collected when unreferenced; structural sharing ensures only unshared portions are reclaimed.

FOL's persistent metaclass adapts the MOP to guard mutation and route overflows. Protocols like `slot-value-using-class` work unchanged for the common case (≤ 8 slots), providing direct $O(1)$ access. For wider objects, `slot-missing` intercepts the access and redirects it to the overflow trie. Crucially, `(setf slot-value-using-class)` is specialized to block any post-initialization writes, signaling an error if the user attempts to move an object out of its immutable state.

This adaptation is mandatory for three reasons. First, persistence cannot be implemented as a library because transparent mutation-guarding demands metaclass-level interception: a library-based approach can provide immutable data structures, but it cannot prevent a user from accidentally applying `setf` to a slot unless it overrides the MOP. Second, the overflow trie mapping is logically separate from the physical slot layout; the MOP allows us to present a unified "slot" interface to the user even when the underlying storage is forked between native slots and a HAMT. Third, the lazy migration strategy for class redefinition (Section 5.4) requires intercepting slot access to transform legacy values on-the-fly.

The engineering of this adaptation centers on three concrete decisions: (1) the hybrid native-slot/overflow-trie layout that preserves $O(1)$ access for the common case; (2) the threshold $T=8$ chosen by cross-referencing empirical class-size data with micro-benchmark crossover analysis (Section 4.6); and (3) the lazy migration strategy for class redefinition that maintains persistence across schema boundaries. By anchoring these in the MOP, we ensure that persistence is deep-rooted in the language semantics rather than being a convenient convention.

The hybrid layout implementation uses a specialized slot-definition class, `persistent-effective-slot-definition`. During the class initialization protocol, the metaclass calculates the slot index. If the index is $\leq T$, it allocates a standard CLOS slot; otherwise, it marks the slot as virtual. When `slot-value-using-class` encounters a virtual slot, it performs a lookup in the HAMT-based overflow trie. This lookup is performed using the slot name's symbol identity

as the key. Because HAMT lookups are $O(\log_{32} N)$, even a class with 1,000 slots maintains extremely fast access. The metadata slot (`%metadata`) is always native, ensuring that introspection logic never recurses into the overflow logic.

3 Features

3.1 Clojure Features

FOL implements key Clojure abstractions:

- **Transducers** [15]: Composable transformation pipelines that decouple the transformation logic from the input source. FOL's transducers are implemented as generic functions over the reducible protocol, allowing them to transform not just native collections but also streams, channels, and custom user-defined data sources.
- **Lazy sequences**: On-demand computation using a delay-based memoization strategy. FOL's lazy sequences are *realized* only once and cached, ensuring that expensive computations are amortized across multiple traversals.
- **Atoms/Refs**: Concurrency primitives for managed mutable state. Atoms provide uncoordinated, synchronous change via compare-and-swap (CAS), while Refs provide coordinated, synchronous change via software transactional memory (STM).
- **Sequence functions**: Over 100 functions including `map`, `filter`, `reduce`. These are implemented as generic functions that automatically dispatch to the most efficient implementation for the given collection type (e.g., array-based indexing for vectors vs. pointer-chasing for lists).
- **Macros**: Clojure-style `defmacro` with `syntax-quote` and `unquote`. This allows for powerful syntactic abstractions that are expanded at compile-time into native Common Lisp.

Transient builders provide a mutation-windowing protocol that reduces functional-update costs from $O(k)$ HAMT path copies to $O(1) + O(\log n)$ for batch writes. To ensure safety, FOL enforces *thread-confined runtime guards*: the transient call returns a shallow copy with a volatility token set to the *current thread ID*; further mutations (`assoc!`) verify this token and signal an error if the object is accessed from a different thread or has already been frozen via `persistent!`. FOL additionally provides `transfer-ownership!` for linear handoffs in actor-pipeline patterns, where Thread A builds a transient and passes it to Thread B for further enrichment before freezing. The call atomically replaces the owner token with the new thread's identity—simultaneously invalidating the original thread's access so that any subsequent `assoc!` or `persistent!` by the original thread signals an error. This is a runtime approximation of *"uniqueness types"* [2, 20]: it provides use-after-transfer detection rather than static prevention. This pragmatic choice catches concurrency errors during development without the verification burden of a full linear type system.

3.2 Generic Function Integration

FOL's generic functions dispatch by arity first, then by type predicates and specificity within an arity group. Methods at different arities may coexist on the same generic:

```

233 (defgeneric process [x])
234 (defmethod process
235   ([x <vector>]) (vec (map process x)))
236   ([x <dict>])   (update-vals x process))
237   ([x <number>]) (* x 2)))
238 (process [1 {a 2 b 3} [[4]]]) ; => [2 {a 4 b 6} [[8]]]

```

Multi-clause `defmethod` forms expand to separate method definitions. The `:before`, `:after`, and `:around` qualifiers apply per-clause; `call-next-method` follows CLOS semantics.

defn vs defgeneric/defmethod: Every FOL function is dispatched by specificity. `defn` defines a "closed" multi-clause function where all clauses are colocated, similar to pattern-matching in Haskell or Clojure's `defn`. This is the idiomatic choice for fixed domain logic. The FOL compiler optimizes `defn` by emitting a single `defun` that performs an efficient cond-based routing over the provided patterns. Because the patterns are known at compile-time, the compiler can often statically eliminate type-checks or optimize destructuring into direct array accesses.

In contrast, `defgeneric` and `defmethod` provide "open" extensible dispatch in the CLOS tradition, allowing different modules to add clauses to the same function. This extensibility comes with a performance trade-off: because the set of methods is open-ended, the system must perform a dynamic specificity scan on every invocation. Qualifiers like `before`, `after`, and `around` are restricted to `defmethod` to preserve the performance of `defn` as a simple, high-speed routing form. FOL's `defn` is thus a performance-optimized subset of its full generic function machinery.

Figure 1 gives the formal grammar for FOL's multi-pattern dispatch syntax.

Note that *pattern* forms defined in the grammar lack a body (used for `defgeneric` declarations), while *clause* forms combine a *pattern* and evaluating *body* (used for implementations).

FOL extends pattern matching with predicate specializers. The syntax `(var (fn args...))` evaluates `(fn var args...)` at runtime. Patterns are ordered by *constraint strength* across four syntactically recognizable levels (Table 1), using definition order as a tie-breaker. This *syntactic precedence model* avoids the undecidability of logical subsumption by using the observable form of the specializer as a proxy for narrowness. LEX comparison of level vectors ensures left-to-right scanning dominance.

Table 1: Predicate Specificity Categories

Level	Category	Example
3	Predicates	<code>(x (even?))</code>
2	Types	<code>(x <integer>)</code>
1	Destructuring	<code>[x y]</code>
0	Wildcard	<code>x</code>

3.2.1 Formal Specificity Ordering. We formalize the specificity relation $\prec$ using the inference rules summarized in Table 2. Let $\mathcal{L}(p) \in \{0, 1, 2, 3\}$ be the specificity level of pattern p .

The operational dispatch order $\prec_{\text{disp}}$ resolves ties using definition order. Let $\text{idx}(p)$ denote the definition index of the method clause

Table 2: Specificity Inference Rules

Rule	Formalization
Spec-Level	$\frac{\mathcal{L}(p_1) > \mathcal{L}(p_2)}{p_1 \prec p_2}$
Spec-Prefix	$\frac{p_1 \prec q_1}{[p_1, \dots] \prec [q_1, \dots]}$
Spec-Tail	$\frac{\mathcal{L}(p_1) = \mathcal{L}(q_1) \quad [p_2, \dots] \prec [q_2, \dots]}{[p_1, \dots, p_n] \prec [q_1, \dots, q_n]}$
Spec-Length	$\frac{\mathcal{L}(p_i) = \mathcal{L}(q_i) \quad \forall i \in \{1 \dots n\}}{[p_1, \dots, p_n, p_{n+1}] \prec [q_1, \dots, q_n]}$
Spec-Fixed	$\frac{\mathcal{L}(p_i) = \mathcal{L}(q_i) \quad \forall i \in \{1 \dots n\}}{[p_1, \dots, p_n] \prec [q_1, \dots, q_n \ \& \ r]}$
Spec-Map	$\frac{p \prec q}{\{k \ p, \dots\} \prec \{k \ q, \dots\}}$
Spec-Keys	$\frac{\text{keys}(q) \subset \text{keys}(p)}{p \prec q}$

containing p .

$$p_1 \prec_{\text{disp}} p_2 \iff p_1 \prec p_2 \vee (\neg(p_2 \prec p_1) \wedge \text{idx}(p_1) < \text{idx}(p_2))$$

Patterns of different arity are incomparable; dispatch selects the arity group first, then applies total order $\prec_{\text{disp}}$. Lexicographic levels ensure $p \prec_{\text{disp}} q$ or $q \prec_{\text{disp}} p$ for all same-arity patterns. While definition-order tie-breaking is a modularity hazard, it follows Lisp's interactive workflow.

LEMMA 3.1 (TOTALITY). $\prec_{\text{disp}}$ is a strict total order on same-arity patterns.

PROOF SKETCH. Every pattern position p_i is mapped to a level $\mathcal{L}(p_i) \in \{0, 1, 2, 3\}$. The specificity $\prec$ induces a total lexicographic order on the vectors $(\mathcal{L}(p_1), \dots, \mathcal{L}(p_n))$. For any two patterns p, q , if their level vectors differ, they are strictly ordered. If they are identical (a tie), definition order $\text{idx}(p)$ is a total order on the source text, ensuring $p \prec_{\text{disp}} q$ or $q \prec_{\text{disp}} p$. $\square$

Predicate dispatch requires an $O(N)$ linear scan per invocation as resolution cannot be statically cached. While this is conventionally seen as a performance drawback, it provides a predictable upper bound for real-time applications unlike theorem-prover-based dispatch which can trigger unpredictable latency. To mitigate this cost, FOL's generic function metaobjects partition methods by arity group. This ensures that a call to a 2-argument function only scans the subset of methods with matching arity, greatly reducing N in most production workloads.

To further optimize this scan, FOL adopts a technique termed **dispatch stratification**. Methods are partitioned into two strata: the *static stratum* (Levels 0–2) containing methods specialized only on types, wildcards, or structural destructuring; and the *dynamic stratum* (Level 3) containing methods with arbitrary predicates. Because Levels 0–2 are stable and decidable, the static stratum can be pre-compiled into a standard CLOS dispatch table or decision tree. The dynamic stratum is only scanned if the static stratum fails

<code>defn</code>	<code>::= (defn name (single-clause multi-clause))</code>	
<code>defgeneric</code>	<code>::= (defgeneric name (single-pattern multi-pattern))</code>	
<code>defmethod</code>	<code>::= (defmethod name qualifier[?] (single-clause multi-clause))</code>	
<code>fn</code>	<code>::= (fn (single-clause multi-clause))</code>	
<code>single-clause</code>	<code>::= pattern body⁺</code>	
<code>multi-clause</code>	<code>::= (pattern body⁺)⁺</code>	
<code>single-pattern</code>	<code>::= pattern</code>	
<code>multi-pattern</code>	<code>::= pattern⁺</code>	
<code>pattern</code>	<code>::= [param*] [param* & rest-param]</code>	
<code>param</code>	<code>::= symbol</code>	(simple binding)
	<code> (symbol type)</code>	(type specifier)
	<code> (symbol (fn-name arg*))</code>	(predicate specifier)
	<code> destructure</code>	(nested destructuring)
<code>destructure</code>	<code>::= [param*]</code>	(sequential)
	<code> { :keys [key-spec*] }</code>	(map)
<code>key-spec</code>	<code>::= symbol (symbol type) (symbol pred)</code>	
<code>type</code>	<code>::= <type-name></code>	
<code>qualifier</code>	<code>::= :before :after :around</code>	

Figure 1: FOL Multi-Pattern Dispatch Grammar (simplified)

to find a match or if a more-specific predicate method exists. Type predicates like `<number?>` are recognized syntactically (Level 2), allowing the compiler to optimize them as native type-checks. While this syntactic boundary is a pragmatic compromise for decidability, it breaks referential transparency—aliasing a predicate shifts its level—and metadata-based markers (Section 5.6) are the intended architectural resolution. `call-next-method` within specialized auxiliaries follows standard CLOS precedence, resolving the next most specific method according to the total order $\prec_{\text{disp}}$.

FOL’s persistence simplifies complex state patterns. **Structural Diff** (Listing 1) uses an `:around` method on `assoc` to detect changes by comparing snapshots, triggering automatic audit logs. **Speculative Execution** enables rejecting updates by returning the pre-update snapshot (`obj`), which fails in mutable CLOS as `call-next-method` modifies state in-place. **Content routing** via predicate dispatch (Listing 2) allows sensors to trigger alerts based on data values without manual cond cascades.

Listing 1: Automatic Structural Diff

```
(defmethod assoc :around [(obj <diffable>) key val]
  (bind [old (get obj key)
        new-obj (call-next-method)]
    (if (= old (get new-obj key))
        new-obj
        (assoc new-obj :_changes (inc (get obj :_changes 0))))))
```

Listing 2: Value-Based Alerting

```
(defmethod notify [{:keys [temp]} (>= 100)])
(print "ALERT: Critical Temperature!")
```

Standard CLOS redefinition assumes mutable instances. FOL instead uses *lazy schema evolution*: instances remain frozen Schema 1 snapshots, while functional updates (`assoc`) perform implicit migration by allocating Schema 2 instances and copying old slots forward. `:alias` annotations (Listing 3) allow renamed slots to pull legacy data during migration. Single-hop recursive lookups

follow alias chains, and conflicting aliases are handled by standard initialization order.

Listing 3: Lazy Schema Migration with Aliasing

```
;; Schema 1
(defclass <sensor> [] [[id] [reading]])
;; Schema 2: rename reading -> val, keep history
(defclass <sensor> [] [[id] [val :alias reading]])
```

4 Evaluation

4.1 Comparison with Related Languages

Table 3: Built-in Feature Comparison

Feature	FOL	Clojure	CL
Persistent objects	✓	✓	×
CLOS/MOP	✓	×	✓
Method combinations	✓	×	✓
Predicate dispatch	✓	single ^a	×
Transducers	✓	✓	×
Lazy seqs	✓	✓	×
Macros	✓	✓	✓

^a `defmulti` provides single-parameter predicate dispatch (arbitrary fn) but lacks multi-parameter specificity ordering.

^b Multi-parameter type dispatch natively; libraries like `trivia` add pattern matching.

FOL uniquely combines Clojure’s functional features with Common Lisp’s object system. Persistent objects with fewer than 8 slots maintain $O(1)$ access and mutable-equivalent update cost because they bypass the overflow trie entirely. Those with more than 8 slots and most collections incur $O(\log n)$ vs. $O(1)$ for mutable operations, but high branching factors (32–64) keep constant factors small. The performance boundary of FOL is thus defined not by algorithm complexity, but by the constant-time multiplier of persistence.

Listing 4: Naive Persistent Partition

```

(defn partition [v low high]
  (bind [pivot (get v high)]
    (loop [j low i (dec low) curr-v v]
      (if (< j high)
        (if (<= (get curr-v j) pivot)
          (bind [next-i (inc i)]
            (v1 (assoc curr-v next-i (get curr-v j)))
            (v2 (assoc v1 j (get curr-v next-i))))
          (recur (inc j) next-i v2))
        (recur (inc j) i curr-v))
      (bind [next-i (inc i)]
        (v1 (assoc curr-v next-i (get curr-v high)))
        (v2 (assoc v1 high (get curr-v next-i)))
        [v2 next-i]))))

```

Our implementation of transducers allows them to target any collection type via a unified reducible protocol. Unlike Clojure, which relies heavily on Java’s `Iterable` interface, FOL’s transducers are generic functions that can be extended by the user to support third-party data structures without modifying the core library. This extensibility is a direct benefit of the MOP integration: we can add new collection behaviors by specializing protocols at the metaclass level.

4.2 Benchmark Methodology

All measurements were taken on SBCL 2.6.0 (Windows 11, AMD Ryzen 9 5900X, 64 GB RAM). Feature benchmarks report the mean of 100 runs (coefficient of variation <2%); LSim benchmarks report the mean of 10 runs. For timing, we use `get-internal-run-time`; for allocation, we use SBCL’s `(sb-ext:get-bytes-consed)`.

We compare against native Common Lisp rather than Clojure for several reasons. First, SBCL compiles to native code, providing a strict upper bound on overhead that isn’t masked by JVM jit-warmup or garbage collector abstractions. Second, since FOL transpiles to Common Lisp, this isolates the cost of the language abstraction itself from the cost of the underlying runtime. Third, Clojure’s object orientation is fundamentally different (protocol-based rather than class-based), whereas FOL’s use of CLOS allows for a direct comparison of the “persistence tax” on the exact same object model.

4.3 Translation Penalty Analysis

Functional slot updates are 33–447× slower than in-place (`setf slot-value`) (quantified in Section 4.6), defining FOL’s operating boundary for imperative workloads. FOL is structurally unsuitable for algorithmic workloads dominated by tight mutation loops *unless* the transient-builder protocol (Section 3.1) is used: transients batch multiple slot writes into a single allocation, reducing the cost of bulk updates to mutable-equivalent construction.

We evaluated naive persistent translations of quicksort and BFS to isolate the path-copying penalty; each `assoc` recreates the HAMT path to the root, generating intermediate nodes on every swap or visited vertex. Listing 4 shows the naive partition logic.

Table 4 shows results for all three implementations on both workloads (SBCL 2.6.0, AMD Ryzen 9 5900X, 3 runs each).

Listing 5: Naive Persistent BFS Step

```

(defn bfs-step [q dists graph]
  (if (empty? q)
    dists
    (bind [u (first q)]
      (edges (get graph u)
        d (get dists u)]
      (loop [es edges nq (rest q) nd dists]
        (if (empty? es)
          (bfs-step nq nd graph)
          (bind [v (first es)]
            (if (= (get nd v) -1)
              (recur (rest es)
                (conj nq v)
                (assoc nd v (inc d))))
              (recur (rest es) nq nd)))))))

```

Table 4: Adversarial Workload Results

Workload	Implementation	Time	Alloc	vs. CL
QuickSort $N = 100,000$	CL (simple-vector)	23 ms	0.8 MB	1.0×
	FOL Persistent (assoc)	1310 ms	2947 MB	57.5×
	FOL Transient (assoc!)	474 ms	91 MB	20.8×
BFS $N = 50,000$	CL (hash-table)	2 ms	2.5 MB	1.0×
	FOL Persistent (assoc)	52 ms	53 MB	28.3×
	FOL Transient (assoc!)	11 ms	8.2 MB	5.8×

For the 100,000-element quicksort, naive persistent FOL generates ~3.4 million HAMT path-copy allocations—two `assoc` calls per swap—producing 2,947 MB of garbage at 57.5× the CL time. For the 50,000-node BFS, naive persistent FOL incurs one `assoc` per visited node, generating 53 MB at 28.3×. The transient-builder protocol substantially recovers performance in both cases: transient quicksort reduces overhead to 20.8× with 97% less allocation (91 MB); transient BFS reduces overhead to 5.8× with 85% less allocation (8.2 MB), as `hamt-assoc!` mutates the distance map in place across the traversal before freezing the result with `hamt-persistent!`. These results establish a “translation penalty” for imperative algorithms; they do not define the language’s expressivity boundary, as an idiomatic FOL developer would use a persistent radix sort or a functional graph traversal that minimizes allocation through structural sharing.

The stark difference between naive persistence and transient-based updates highlights that the primary performance bottleneck in FOL is not the $O(\log N)$ algorithmic complexity of path-copying, but rather the memory-subsystem pressure caused by repeated intermediate allocations. In the naive QuickSort case, each swap generates multiple intermediate HAMT root nodes that are immediately discarded. The transient-builder protocol addresses this by allowing a sequence of mutations to occur within a single, thread-confined allocation window. This effectively “fuses” multiple functional updates into a single persistent result, bringing the cost of bulk updates much closer to the cost of mutable array manipulation. For the BFS workload, the 5.8× overhead of the transient version is particularly revealing: it suggests that once allocation costs are managed, the overhead of HAMT-based indexing is roughly five

times that of a native hash table—a ratio that matches the depth of the 32-way trie branching.

4.4 Feature Benchmarks

We evaluated FOL across three benchmarks exercising specific language features: structural diff, hierarchical speculative execution, and observable slots. Table 5 summarizes performance and memory ratios.

Table 5: Feature Benchmark Summary

Benchmark	Perf Ratio	Mem Ratio
Structural Diff	113.7×	55.3×
Hierarchical Speculative Execution	23.4×	9.0×
Observable Slots	6.69×	6.32×

Structural Diff (500,000 iterations, 5-slot <metric> object): Each iteration makes six assoc calls—one `_changes` reset followed by five slot updates. Each call dispatches through the `fol.core:assoc` generic function, invokes the `<diffable> :around` method (reading the pre-update value and the post-update result), and when a change is detected triggers a *recursive* assoc to increment `_changes`—three HAMT path copies per changed slot. The 113.7× time overhead is compounding: CLOS dispatch, `:around` trampoline, HAMT path copy, and recursive re-entry all multiply. The 55.3× memory ratio reflects six HAMT copies per iteration plus recursive increments, versus a single struct allocation in the CL baseline ($\approx 3,370$ bytes vs. 61 bytes per iteration).

The memory overhead is *inherent* to the design: each assoc call must produce a fresh immutable snapshot. Production code replacing the six sequential assoc calls with a single `update-slots` batch would eliminate five of the six intermediate allocations; the benchmark uses sequential calls to isolate per-call overhead.

Hierarchical Speculative Execution (1,000,000 deposits through a three-level `:around` chain): The 23.4× overhead matches the single-guard case, confirming that the dominant cost is the HAMT path copy in the persistent-object update, not the method-combination chain depth. SBCL’s PCL compiles the effective method for the three `:around` layers once and caches it; the chain adds no additional dispatch cost. The 9.0× memory ratio reflects one HAMT path copy per deposit (≈ 412 bytes vs. 46 bytes).

Observable Slots (1,000,000 sensor-update iterations): The 6.69× overhead reflects the cost of producing two persistent maps per iteration—the updated sensor state and the change event record. The 6.32× memory overhead stems from two HAMT allocations per update rather than the shallow struct copy of the CL baseline.

The feature benchmarks show overhead ratios from 6.69× to 113.7×, scaling with the number of assoc calls per logical operation. These ratios represent the fundamental cost of FOL’s persistent object model; for workloads that require structural integrity, audit history, or safe rollback, the overhead is a consequence of correctness-by-construction rather than incidental implementation cost. Syntactic abstraction overhead is minimized by the compiler, which emits direct defun calls for internal function definitions. Remaining non-object overhead stems from closure allocation and

runtime dispatch checks for late-bound or external symbols (where a call must support fallback to collection access).

4.5 Logic Simulation (LSim)

LSim is a digital logic simulator that exercises persistent state management at scale. We evaluated four configurations of increasing complexity: 8bit-100 (small), 32bit-300 (medium), 8x32-900 (a 1024-gate pipeline of eight cascaded 32-bit registers), and 32x32-3000 (a 4096-gate pipeline of thirty-two cascaded 32-bit registers with 3000 time steps). Table 6 presents the results.

Table 6: LSim Scaling Results

Config	CL	FOL	Ratio
8bit-100	3 ms	22 ms	7.0×
32bit-300	14 ms	170 ms	12.1×
8x32-900	1.02 s	4.26 s	4.2×
32x32-3000	48.3 s	61.3 s	1.27×

The performance ratio narrows from 7.0× to 1.27× as circuit complexity grows. This trend reflects amortization rather than elimination of overhead: the absolute cost of persistent allocation remains, but it becomes a smaller fraction of total runtime as the simulation computation grows. At the largest scale (32x32-3000, ~ 4096 NAND gates, 3000 time steps), simulation work dominates and the persistent data structure overhead accounts for only 27% of total execution time.

The LSim simulation is implemented using an event-driven model where each NAND gate is represented as a persistent FOL object. Signal propagation triggers a cascade of assoc calls as gate outputs transition between logic states. In a traditional mutable implementation, each transition overwrites the gate’s state, losing the history of the signal’s propagation unless a global snapshot is explicitly taken. Crucially, in contexts like simulation where state history must be preserved for analysis or event-replay, a mutable CLOS implementation must defensively deep-copy the entire circuit state at each time step (an $O(N)$ allocation). FOL’s structural sharing allocates only $O(\log n)$ memory for the changed branches. At scale, this structural sharing inverts the naive expectation: GC pressure benchmarks for the 32x32-3000 circuit show that FOL allocates only 0.59× the bytes of the Common Lisp version and triggers 0.59× as many GC invocations. This empirical result confirms that at the limits of state scaling, the efficiency of structural sharing outweighs the overhead of persistent allocation. The persistent model essentially provides a “free” undo-log and history mechanism that would otherwise require manual, high-overhead management in a mutable system.

4.6 Persistent Object Micro-Benchmarks

To isolate the cost of FOL’s persistent metaclass from higher-level transpiler overhead, we measured construction, read, and functional update costs for persistent objects versus standard mutable CLOS objects across a range of slot counts. **Construction overhead** is modest (1.67–1.72× memory). **Native slot reads** are efficient (1.4–2.1× mutable CLOS), while overflow reads cost 3–5× due to MOP interception. **Functional updates** are the primary cost (33–447×

slower than `setf`), amortized by batching. Table 7 evaluates the threshold selection: $T=8$ minimizes update cost above $N \approx 28$ slots while keeping the common case on the native path.

The preceding construction figures compare FOL against mutable CLOS. Measured against a lean `defstruct` baseline—the representation a Clojure `defrecord` approximates—two distinct costs emerge. **Retained size:** each FOL object carries two hidden overhead slots (`%persistent-vector`, `%metadata`) on top of user slots, producing objects 2.5× larger than a struct for 1–2-slot classes (the statistical median) and 1.4× at 8 slots. Both the user slots and the two overhead slots are stored in a CLOS slot-vector—a separate heap allocation—giving each object a two-allocation footprint vs. a struct’s one. **Construction GC pressure:** the bytes *consed* per allocation—the true GC pressure signal—are 8–15× higher than an equivalent CLOS construction: a 2-slot FOL object consumes ≈ 416 B/obj during construction vs. 48 B/obj for plain CLOS and 32 B/obj for a struct. The gap between construction cost (288–2,685 B/obj) and retained size (80–112 B/obj) shows that 80–95% of construction bytes are transient garbage from `initialize-instance` MOP machinery—slot iteration, `initarg` keyword dispatch, and internal CLOS caching. This is a secondary explanation for the high allocation ratios in the adversarial benchmarks (Section 4.3): each `assoc` call triggers not just a HAMT path copy but a full `initialize-instance` dispatch. It also explains why the transient-builder protocol reduces allocation by 97%: transients bypass `initialize-instance` entirely during the mutation window, writing directly into the mutable HAMT without object construction overhead.

A separate micro-benchmark evaluated the five candidate thresholds by directly simulating hybrid copy at each T across object sizes $N \in \{8, 12, 16, 24, 32, 48\}$ (Table 7). Persistent vector batch trie construction costs ≈ 106 ns base plus 5–7 ns per element—an order of magnitude less per element than slot-value dispatch (≈ 27 ns/slot). For objects at or below the threshold (all-native path), all candidates are equivalent. The crossover occurs near $N \approx 28$: above this size, $T=8$ minimizes update cost. Objects in the 9–24 slot range incur a modest penalty at $T=8$ (0.12–0.17 μ s, roughly 15–21%) from applying the vector base cost to small overflows. However, empirical OO class-size studies report the 90th percentile at ≈ 10 –12 fields (median 2), placing $\leq 10\%$ of classes in this penalty zone while $\sim 80\%$ remain all-native. Note that the simulated costs in Table 7 represent raw copy work; the actual update-slots costs (Table ??) are 1.6–3× higher due to MOP slot iteration, slot-bound checks, and keyword-to-slot-name lookup overhead.

4.7 Threats to Validity

Several threats to validity should be considered. **Single implementation:** All measurements use SBCL 2.6.0 on a single platform (AMD Ryzen 9, Windows 11); results may differ on other CL implementations or architectures. **Benchmark coverage:** Our evaluation includes both *feature benchmarks* (demonstrating the architectural synergy of persistence and dispatch) and *adversarial workloads* (quantifying the performance penalty of naive persistent algorithms). While this provides a comprehensive view of FOL’s performance envelope, the feature benchmarks naturally select for patterns where FOL excels, and the adversarial benchmarks isolate worst-case overheads that may be mitigated in production through

Table 7: Modeled update cost (μ s/op) at candidate thresholds T

N	$T=8$	$T=12$	$T=16$	$T=24$	$T=32$	$T=8$ strategy
8	0.42	0.42	0.43	0.42	0.42	all-native
12	0.75	0.62	0.62	0.61	0.60	8N + 4V
16	0.94	0.91	0.83	0.78	0.77	8N + 8V
24	1.15	1.23	1.28	1.15	1.15	8N + 16V
32	1.39	1.47	1.51	1.58	1.52	8N + 24V
48	1.95	1.99	2.04	2.12	2.22	8N + 40V

Bold = minimum in row. “ $kN + jV$ ” = copy k native slots then batch-build j -element overflow vector. All-native = copy N slots directly. Simulated copy costs only; actual update-slots is 1.6–3× higher due to MOP iteration overhead (see Table ??).

idiomatic persistent algorithms or the transient-builder protocol. **Warmup effects:** Performance measurements report means of multiple runs, which amortize SBCL compilation and cache warmup; the first-run latency is predictably higher. As a single-author project, FOL has not yet undergone independent code review; the full source is publicly available to facilitate external verification.

FOL is transpiled to Common Lisp, introducing a threat specific to dispatch performance: predicate specializers (Levels 2–3) cannot be cached by class signature and require runtime evaluation on each invocation, so reported dispatch overhead compounds FOL’s routing cost with SBCL’s PCL cache-miss penalty. A native compiler could replace the $O(N)$ scan with an ordered type-check sequence; the dispatch ratios reported here are therefore upper bounds on inherent routing cost.

No static analysis exists—type errors and unreachable clauses are detected at runtime only; some type-dependent code may require runtime resolution, degrading transpiled performance. The compiler has not been extended to warn about shadowed clauses; a future version could detect shadowing statically.

The specificity calculus struggles with compound predicates; conjunctions such as `(and (<number>?) (even?))` fall to Level 3 even if they logically narrow a Level 2 type. This highlights the inherent difficulty of providing a sound, decidable subtyping relationship over arbitrary dynamic logic. A recursive extension— $\mathcal{L}(\text{and } p \ q) = \max(\mathcal{L}(p), \mathcal{L}(q))$ —would lift compound type-predicate conjunctions but cannot order disjoint Level 3 predicates whose containment is undecidable. Definition-order tie-breaking, while a *modularity hazard* where load order determines behavior, aligns with Lisp’s interactive, last-definition-wins workflow. A strict approach signaling ambiguity errors would improve modularity but disrupt the development ergonomics typical of the ecosystem.

The dominant performance limitation remains the functional update cost (Section 4.3). Transient builders (Section 3.1) address initialization-heavy operations by batching slot writes within a delimited scope, reducing the cost to mutable-equivalent construction; the residual overhead for single-slot mutation loops is inherent to the persistent model and requires idiomatic persistent algorithms or acceptance of mutable structures for those paths. Class redefinition does not migrate existing instances (Section ??); old objects retain their original slot data, with new slots resolved lazily via `initforms`.

Planned extensions include **dispatch stratification** (implementing the two-strata model described in Section 4.2 using decision

trees and fast-path caches for Levels 0–2), **metadata-based type markers** to accelerate predicate evaluation, **abstract and sealed classes** to enable better static deviance checking, **parallel collections** that leverage structural sharing for fork-join parallelism, **channel-based concurrency** (CSP-style go-blocks) integrated with the sequence protocol, a **native compiler** targeting LLVM or machine code directly, and a **VSCode extension** for integrated persistence debugging.

5 Related Work

Clojure [13] pioneered persistent data structures in Lisp. FOL adopts its sequence and transducer abstractions but trades protocol-based dispatch for full CLOS/MOP. **Common Lisp**’s CLOS [4] and MOP [16] provide the foundation; while libraries like FSet [6] provide persistence, they lack MOP-integrated metadata management. **Elephant** [11] explored metaclass-based persistence but focused on disk-backed storage rather than structural sharing. **Dylan** [1] inspired FOL’s naming conventions and its pure OO focus. **Predicate dispatch** builds on Cecil [7, 10], whose subsumption-based specificity is more general but undecidable; FOL’s category-based ordered dispatch ensures predictability by inspection. **Racket** [12] offers a distinct synthesis of records and classes; FOL differs by unifying them such that every object *identity* is both a CLOS instance and a persistent record, enabling metaclass-level interception of updates that Racket’s separate struct/class hierarchies cannot easily coordinate.

FOL’s functional update syntax (update-slots) mirrors record updates in **F#** and **Elm** [8]. While **Julia** [3] uses multiple dispatch, it lacks predicate specializers and method combinations. **Coalton** [17] and **Typed Racket** [18] focus on static safety, whereas FOL prioritizes persistence. FOL’s lazy migration (Section 5.4) shares goals with dynamic evolution in **Self** [19] and **Newspeak** [5], but applies them to immutable state: old snapshots remain valid naturally as migration occurs only during functional updates.

6 Conclusion

FOL demonstrates a successful synthesis of persistent data structures and CLOS, yielding architectural patterns like versioned objects and declarative event sourcing that are difficult to achieve in either paradigm isolation. Our evaluation confirms that while persistent allocation introduces significant micro-overhead (up to 113.7×), this cost is effectively amortized in non-trivial workloads, reaching near-parity (1.27×) in large-scale simulations. Crucially, we find that the efficiency of structural sharing actually inverts the

naive expectation of GC pressure at scale: FOL allocates only 0.59× the bytes of a deep-copying mutable equivalent, indicating that persistence is a modular engineering cost that can be managed and even leveraged through principled MOP-level design. This work shows that functional object orientation is a viable and expressive choice for modern, state-heavy systems. The complete implementation is available at: <https://github.com/frankadrian314159/fol>

References

- [1] Inc. Apple Computer. 1992. *Dylan - An Object-Oriented Dynamic Programming Language*. Apple Computer, Inc.
- [2] Erik Barendsen and Sjaak Smetsers. 1993. Uniqueness Typing for Functional Languages. In *Proceedings of the 5th International Workshop on Implementation of Functional Languages*. 1–25.
- [3] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B Shah. 2017. Julia: A Fresh Approach to Numerical Computing. *SIAM Rev.* 59, 1 (2017), 65–98.
- [4] Daniel G Bobrow, Linda G DeMichiel, Richard P Gabriel, Sonya E Keene, Gregor Kiczales, and David A Moon. 1988. Common Lisp Object System Specification. *SIGPLAN Notices* 23, SI (1988), 1–142.
- [5] Gilad Bracha, Peter von der Ahé, Vassili Bykov, Yaron Kishai, William Maddox, and David Ungar. 2010. The Newspeak programming language. In *Proceedings of the 19th symposium on Dynamic languages*.
- [6] Scott L Burke. 2007. FSet: A functional set-theoretic collections library. <https://common-lisp.net/project/fset/>
- [7] Craig Chambers. 1993. The Cecil Language: Design and Performance. In *Proceedings of the Conference on Object-Oriented Programming Systems, Languages, and Applications*. 243–257.
- [8] Evan Czaplicki. 2012. *Elm: Concurrent FRP for Functional GUIs*. Master’s thesis. Harvard University.
- [9] James R Driscoll, Neil Sarnak, Daniel D Sleator, and Robert E Tarjan. 1989. Making Data Structures Persistent. *J. Comput. System Sci.* 38, 1 (1989), 86–124.
- [10] Michael D Ernst, Craig Kaplan, and Craig Chambers. 1998. Predicate Dispatching: A Unified Theory of Dispatch. In *ECOOP’98—Object-Oriented Programming*. Springer, 186–211.
- [11] Ian Eslick and Ben Hyde. 2008. Elephant: A Persistent Object Database for Common Lisp. <https://common-lisp.net/project/elephant/>
- [12] Matthew Flatt et al. 2015. The Racket Manifesto. In *20 Years of the Past and 20 Years of the Future (Snapshots), Dagstuhl Seminar 15451*.
- [13] Rich Hickey. 2008. The Clojure programming language. In *Proceedings of the 2008 symposium on Dynamic languages*. 1.
- [14] Rich Hickey. 2009. Are We There Yet? JVM Languages Summit Keynote. https://github.com/matthiasn/talk-transcripts/blob/master/Hickey_Rich/AreWeThereYet.md
- [15] Rich Hickey. 2014. Transducers. Clojure Blog. <https://clojure.org/reference/transducers>
- [16] Gregor Kiczales, Jim Des Rivieres, and Daniel Gureasko Bobrow. 1991. *The Art of the Metaobject Protocol*. MIT press.
- [17] Robert Smith and Cole Bates. 2023. Coalton: Efficient, Statically Typed Functional Programming on Common Lisp. In *Proceedings of the 16th European Lisp Symposium*.
- [18] Sam Tobin-Hochstadt and Matthias Felleisen. 2008. The Design and Implementation of Typed Scheme. In *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. 395–406.
- [19] David Ungar and Randall B Smith. 1987. Self: The power of simplicity. In *ACM SIGPLAN Notices*, Vol. 22. ACM, 227–242.
- [20] Philip Wadler. 1990. Linear Types Can Change the World!. In *Programming Concepts and Methods*.